the implicit social awareness, teaching social context/culture to machines is challenging. Also, hate speech posts typically contain code-switches, where the post switches between different languages. This also makes the job of automated techniques difficult.

Supervised machine learning (ML) techniques for hate speech detection can be of different types. Classical ML techniques involve feature engineering from the text. Standard textual features such as a bag of words, parts of speech tags, sentiments, emotions, stances, etc, can be extracted from the text and used as features to classifiers. Other classes of techniques include using neural networks for hate speech detection. Neural networks have been widely used for various language tasks such as sentence classification, sentiment analysis, emotion classification, etc. These techniques can be adapted for hate speech detection provided there are a sufficient number of labelled instances for training a deep neural network for this task.

If we want to use neural networks for hate speech detection, we first need to know what type of neural network we want to use, such as a feed forward neural network, a convolutional neural network or a recurrent neural network. Recurrent neural networks have been widely used for various language tasks such as sequence-to-sequence labelling, machine translation, etc. While recurrent neural networks have been useful in capturing long range dependencies, convolutional neural networks can be used to identify whether or not a sentence can be classified as hate speech. Let us look at a simple example of this approach.

The idea is to create a sentence embedding matrix where word embeddings for each of the individual words in a given sentence are concatenated together. This sentence embedding is then fed to a convolutional neural network (CNN), which consists of a two-dimensional convolution layer whose output is fed to a max-pooling layer. At the convolutional layer, a 2D filter is constructed and applied over the sentence embedding matrix. There are typically a number of filters, with different filter sizes. The outputs of the max-pooling layer are concatenated and then fed to a multi-layer perceptron whose output layer is the softmax classifier—a two-class classifier that categorises each sentence as hate speech or not. The loss function used can be the standard softmax cross-entropy loss. Here is a quick outline of the pseudocode for this approach. We only show the convolution, pooling and MLP layers.

```
pooled_outputs = []
for i, filter_size in enumerate(filter_sizes):
            # Convolution Layer
            filter_shape = [filter_size, embedding_size, 1,
num_filters]
            W = tf.Variable(tf.truncated_normal(filter_shape,
stddev=0.1), name="W", trainable=trainable)
            b = tf.Variable(tf.constant(0.1, shape=[num_
filters]), name="b", trainable=trainable)
            conv = tf.nn.conv2d(
                    embedded_sent_expended,
                    W,
                    strides=[1, 1, 1, 1],
                    padding="VALID",
                    name="conv")
            # Apply nonlinearity
            h = tf.nn.relu(tf.nn.bias_add(conv, b),
name="relu")
            # Maxpooling over the outputs
            pooled = tf.nn.max_pool(
                    h,
                    ksize=[1, vector_length - filter_size + 1,
1, 1],
                    strides=[1, 1, 1, 1],
                    padding='VALID',
                    name="pool")
            pooled_outputs.append(pooled)

    # Combine all the pooled features
    num_filters_total = num_filters * len(filter_sizes)
    h_pool = tf.concat(pooled_outputs, 3)
    h_pool_flat = tf.reshape(h_pool, [-1, num_filters_total])

    # Final (unnormalized) scores and predictions
    with tf.name_scope("output"):
        W1 = tf.get_variable(
            "W1",
            shape=[num_filters_total, num_classes],
            initializer=tf.contrib.layers.xavier_
initializer(), trainable=trainable)
        b1 = tf.Variable(tf.constant(0.1, shape=[num_
classes]), name="b1", trainable=trainable)
        scores = tf.nn.xw_plus_b(h_pool_flat, W1, b1,
name="scores")
        predictions = tf.argmax(scores, 1,
name="predictions")

    # Calculate mean cross-entropy loss
    with tf.name_scope("loss"):
        losses = tf.nn.softmax_cross_entropy_with_
logits(logits=scores, labels=input_y)
        loss = tf.reduce_mean(losses) + lambda_l2_reg *
l2_loss
        loss = tf.reduce_mean(loss)

    # Accuracy
    with tf.name_scope("accuracy"):
        correct_predictions = tf.equal(predictions,
tf.argmax(input_y, 1))
        accuracy = tf.reduce_mean(tf.cast(correct_
predictions, "float"), name="accuracy")

    optimizer = tf.train.AdamOptimizer(learning_rate=0.01).
minimize(loss)
```